

From Idea to Co-Creation: A Planner-Actor-Critic Framework for Agent Augmented 3D Modeling

Jin Gao
Massachusetts Institute of Technology
Cambridge, Massachusetts, USA
gaojin@mit.edu

Saichandu Juluri
Northeastern University
Boston, Massachusetts, USA
juluri.saichandu99@gmail.com



Figure 1: Multi-Agent Co-Modeling Interface

Abstract

We present a framework that extends the Actor-Critic architecture to creative 3D modeling through multi-agent self-reflection and human-in-the-loop supervision. While existing approaches rely on single-prompt agents that directly execute modeling commands via tools like Blender MCP, our approach introduces a Planner-Actor-Critic architecture. In this design, the Planner coordinates modeling steps, the Actor executes them, and the Critic provides iterative feedback, while human users act as supervisors and advisors throughout the process. Through systematic comparison between single-prompt modeling and our reflective multi-agent approach, we demonstrate improvements in geometric accuracy, aesthetic quality, and task completion rates across diverse 3D modeling scenarios. Our evaluation reveals that critic-guided reflection, combined with human supervisory input, reduces modeling errors and increases complexity and quality of the result compared to direct single-prompt execution. This work establishes that structured agent self-reflection, when augmented by human oversight and

advisory guidance, produces higher-quality 3D models while maintaining efficient workflow integration through real-time Blender synchronization.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; *Human computer interaction (HCI)*; • **Computing methodologies** → *Intelligent agents*; Artificial intelligence.

Keywords

Agent-based systems, Human-in-the-loop, Co-creation, 3D modeling, Interactive AI, Creative tools

ACM Reference Format:

Jin Gao and Saichandu Juluri. 2026. From Idea to Co-Creation: A Planner-Actor-Critic Framework for Agent Augmented 3D Modeling. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3772363.3798904>



This work is licensed under a Creative Commons Attribution 4.0 International License. *CHI EA '26, Barcelona, Spain*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2281-3/26/04

<https://doi.org/10.1145/3772363.3798904>

1 Introduction

Language-based AI agents have rapidly evolved from simple, prompt-driven tools into versatile systems capable of autonomous task execution, including reading, creating, and modifying files; debugging code; performing internet and database searches; and operating desktop applications. Recent interoperability standards such as the Model Context Protocol (MCP) and the Agent-to-Agent (A2A) protocol [4] further enhance these capabilities by standardizing tool usage, structuring information exchange, and enabling collaboration across platforms. Building on these extended agent capabilities, multi-agent systems assign specialized roles to agents that work together via structured communication, which shows enhanced scalability, accuracy, and resilience, especially in complex tasks[6].

While current large-scale neural networks can generate high-quality 2D and 3D frames or models via text-to-3D and image-to-3D prompts, single-shot generations often fall short of user expectations[8]. These outputs typically require manual human evaluation and iterative optimization to progressively approach the desired result[15]. In traditional 3D modeling workflows, humans decompose a model into components and organize them using non-destructive structures like groups, blocks and layers to maximize edit potential. However, mainstream text-to-3D models produce unified meshes, which are difficult to edit. An alternative method control traditional 3D modeling software using APIs, enabling full modeling flexibility including lighting, textures, and third party assets. Meanwhile, evaluating 3D models remains a challenge. Recent advances in multimodal vision-language models (VLMs) have enabled AI systems to interpret and evaluate visual inputs, offering new potential for automated model assessment [13].

Inspired by the actor-critic architecture from reinforcement learning [10, 14], we hypothesize to enhance the quality of creative 3D modeling by integrating planning, action, and critique agents within a multi-agent workflow. Building on this, we further investigate the potential of incorporating human feedback during the agents' workflow, to assess whether human-in-the-loop supervision can provide additional improvements in the modeling outcome.

2 Related Work

Recent work has explored using large language models as agents to directly create or manipulate 3D content. *SceneCraft* [7] introduced an LLM agent that plans a scene graph and then generates Blender Python scripts to construct complex scenes, letting a vision-language model self-critique the result. *WorldCraft* [11] pioneered a collaborative agent system for scene design. It uses a "coordinator agent" to manage two specialist LLM agents – *ForgeIt*, focuses on constructing and customizing individual 3D objects, and *ArrangeIt*, handles spatial layout. This approach outperforms prior single-agent methods in both versatility and user control. Endeavors also focus on fine-tuning domain expert agents to improve control quality. *BlenderLLM* [3] is a domain-tuned LLM that converts natural language instructions into Blender's python (bpy) scripts, improving the reliability of generated 3D scripts over base models.

The general paradigm of *LLM-as-Critic* has proven effective in improving generation quality. In language tasks, *Self-Refine* [12]

and *DeepCritic* [16] showed that iteratively critiquing and revising an LLM's own outputs yields higher performance gains over one-shot answers. In graphics generation, *CoherentDream* [9] uses a fine-tuned vision-language model (*3DLaVA-Critic*) to evaluate the alignment between a generated 3D scene and the textual prompt, achieving state-of-the-art text alignment in multi-object 3D generation. These works demonstrate that a critic agent can lead to higher-quality outcomes.

Finally, integrating human feedback into the LLM loop addresses the "humanity" side of agentic systems. The instability of autonomous systems can struggle with subjective criteria like aesthetics or may fall into infinite loops or premature convergence, making human supervision necessary. *Sel3DCraft* [15] replaces blind trial-and-error prompting with a guided visual interface, helping users identify better results from multi-batch generations.

3 Methodology

3.1 Planner-Actor-Critic Agent Workflow

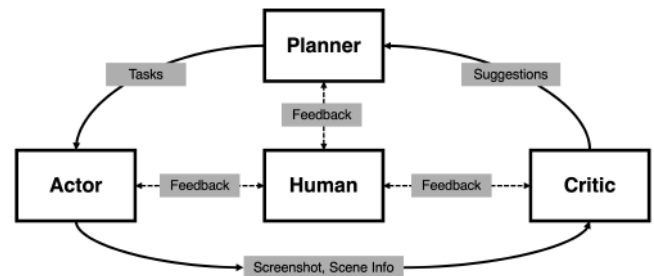


Figure 2: Agent-Human Co-creation Loop

We decompose the modeling task into a collaborative workflow involving three specialized agents. This design is inspired by the workflow of coding agents, particularly Claude Code [2], which maintains a structured plan within the context, maintain task continuity and mitigate issues such as context contamination and attention diffusion, which are common in long, multi-step generation tasks[5]. In our multi-agent framework:

Planner Agent is responsible for proposing and maintaining a list of modeling steps (i.e., a todo list) based on the given task description. This list is persistently stored in a shared context and is continuously updated—steps can be added, modified, or removed based on the Actor's execution results and the Critic's feedback.

Actor Agent autonomously selects a task from the todo list and executes the corresponding action using tools from a predefined toolset. These tools include operations such as primitive creation, modifier application, and model import from local or online libraries. While Blender-MCP already supports several MCP tools, we further extended its toolset to cover common modeling operations, such as creating basic geometry.

Critic Agent evaluates the current modeling result by analyzing the scene state and visual feedback (e.g., screenshots). It then generates structured critique and suggestions, which are fed back to the Planner Agent to edit the todo list.

In our prototype, the agent operates in a unidirectional sequence, progressing from planner to actor to critic, supported by a partially

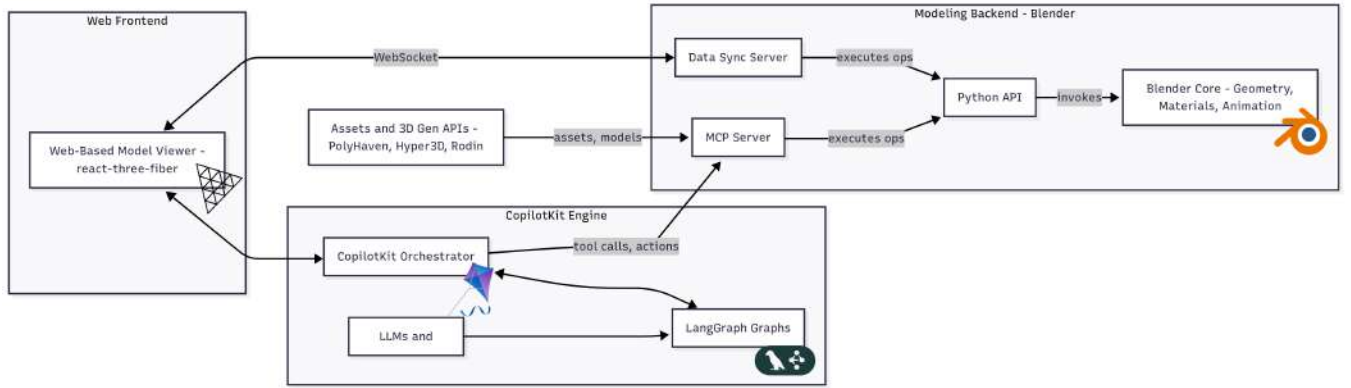


Figure 3: System Diagram

shared information board. A human participant may engage at any stage of the agent dialogue through a dedicated input interface, allowing them to direct, accept, or intervene in the generating process at the agent’s action stage.

3.2 Tooling and Integration

We use Blender-MCP[1] as the backend system for real-time command execution in Blender. The current version of Blender-MCP supports a variety of MCP tools, including executing Python code, retrieving scene information, and downloading models from online 3D asset libraries.

In our interface prototype, tools are organized into toolsets — such as modeling tools and refinement tools — that can be dynamically enabled, disabled, or switched during the modeling process. These toolsets are exposed to both agents and human users, allowing fine-grained control over the modeling workflow and enabling more precise human-agent collaboration.

3.3 System Overview

We selected Blender as the backend platform for testing 3D modeling due to its open-source nature, comprehensive functionality, and robust Python API. Blender offers extensive support for model import/export, modeling operations, material editing, and animation workflows. Additionally, Blender-MCP, a rapidly evolving open-source project, enables scene inspection, automated screenshots, and execution of Python code directly within Blender. It also supports integration with several popular 3D model generation APIs and asset libraries (e.g., PolyHaven, Hyper3D and Rodin), making it suitable for implementing basic agent-environment interaction functionalities.

To enhance the interactivity between Blender and the agent framework, we chose to implement a web-based interface for both agent interaction and real-time 3D model visualization and editing. In this setup, Blender serves as the backend, responsible for core modeling functionalities. We designed and implemented a real-time, bidirectional communication pipeline between Blender and a React-based frontend. The key components include:

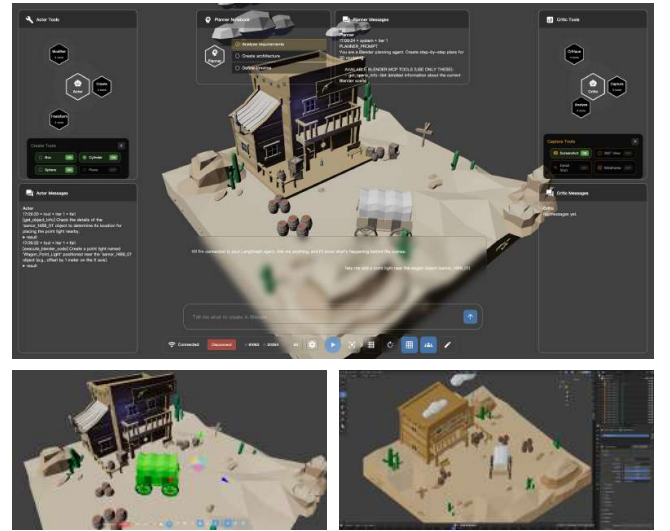


Figure 4: Multi-Agent Co-Modeling Interface and Interaction Examples. Top: main co-modeling interface. Bottom: synchronized viewport and wireframe visualization.

- (1) an extended MCP server and Blender plugin based on the Blender-MCP project,
- (2) a WebSocket-based communication module enabling bidirectional exchange of geometry, lighting, and texture data between Blender and the frontend,
- (3) a multi-agent engine powered by CopilotKit, with LangGraph serves as agent chain-of-thought framework, with OpenAI ChatGPT 5.1 as the base model,
- (4) a unified, web-based user interface.

Together, these components form a full-stack, agent-driven interactive modeling system that enables seamless collaboration between AI agents, human users, and the 3D modeling backend.

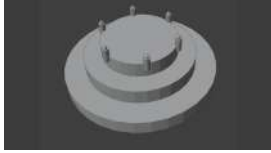
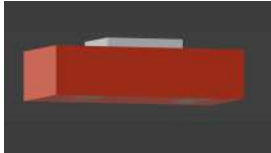
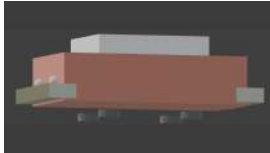
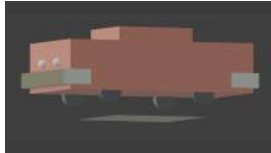
Task	Scale	Initial Prompt	Iter. 1	Iter. 3	Iter. 5
Cake	Small	Model a low-poly birthday cake with visible layers and candles.			
Table	Medium	Create a low-poly square dining table with four legs.			
Car	Large	Construct a low-poly classic car using only primitive shapes.			

Table 1: Selected Modeling Results on Different Tasks.

4 Experiments and Evaluation

4.1 Tasks and Setup

We curated a set of modeling tasks varying in both scale and type, designed to test the system’s adaptability and generalization. To standardize evaluation criteria and reduce the complexity of modeling operations, we adopted a low-poly style across all asset collection and model generation processes. Experiments were conducted on a MacBook Pro equipped with an Apple M4 Pro processor and 48 GB of memory.

4.2 Evaluation Metrics

We employ a combination of automated and human-in-the-loop evaluation methods across three primary dimensions: **Geometric Quality**, **Visual Quality**, and **Human-in-the-Loop Control**. Given the absence of ground truth 3D models for open-ended design tasks, the evaluation follows a hybrid qualitative–quantitative strategy.

Geometric Quality. Geometric quality is assessed using three indicators. *Geometry count* measures the number of distinct objects in the scene after each modeling task. *Vertex count* captures overall mesh complexity at each stage, ensuring adherence to low-poly constraints by avoiding both excessive detail and over-simplification. *Similarity* quantifies changes between consecutive modeling iterations, reflecting the extent of scene modification over time.

Visual Quality. Visual quality is evaluated using a 5-point Likert scale by three independent human raters. Two criteria are considered: *task alignment*, which assesses how well the generated model

fulfills the specified objective in terms of completeness and functional accuracy, and *aesthetic quality*, which evaluates visual appeal based on proportional balance, stylistic consistency, simplicity, and realism. In our small-scale experiments, task alignment was positively correlated with aesthetic quality; therefore, we average the two scores to obtain an overall visual quality metric.

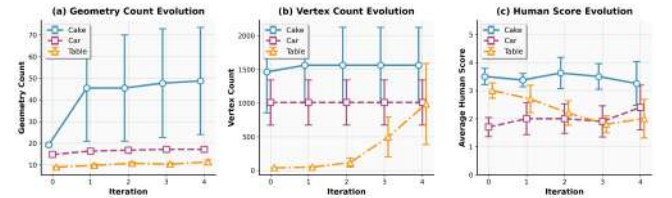


Figure 5: Geometric Quality Analysis in Autonomous Modeling Tasks

5 Discussion & Future Works

5.1 Efficacy of the Self-Reflective Framework

Across many small to medium-scale modeling scenarios, our self-reflective planner–actor–critic framework achieved higher modeling quality than a single-agent baseline, as shown in the first iteration. Improvements were most evident in geometric proportions, fine-grained adjustments, and material assignment. We observe the planner agent decomposes multi-step modeling goals into executable subtasks that can be completed with short code snippets, which reduces hallucinations and error rates. Therefore, the actor

execute more reliable in more complex scenes. The critic inspects scene metadata and screenshots to generate a target-relevance score, influencing the planner’s next decision. Our finding indicate the self-reflective planner–actor–critic pipeline is a viable avenue for automated modeling.

5.2 Emergent Behaviors and Human Oversight

Human participation increases controllability and helps the system escape local optima. In practice, users can introduce creative directives during execution, such as switching a square table to a round table, or change materials to a wood color. This human-in-the-loop reduce failure and inject essential human creativity in the agent-augmented modeling.

5.3 Limitations and Future Improvements

There are several limitation of our current system. First, due to the limitation of total amount of tools allowed in MCP usage, Python scripts driving Blender can fail due to syntax or logic errors. Second, the actor does not always incorporate the critic’s feedback, and after multiple iterations (often three), the modeling quality degrades or converge, making further improvements difficult. Third, the code execution approach often rely on modeling with primitive, which limit expressiveness. Finally, while human intervention improves outcomes, its impact can be constrained or ignored by agents.

In future work, we will refine the MCP tool reliability by boarden types of tools, and introduce pre-execution validation and automatic repair/retry in the workflow. We will fine-tune prompt and enhanced human-in-the-loop control to better guide the agent convert idea to models.

References

- [1] Ahuja, Siddharth and Blender-MCP Contributors. 2025. *Blender-MCP: Blender–Claude AI Integration via MCP*. <https://github.com/ahujasid/blender-mcp> Accessed: 2025-08-10.
- [2] Anthropic. 2024. Claude Code: AI Coding Assistant. <https://www.anthropic.com/claude/code>. Accessed: 2025-08-10.
- [3] Yuhao Du, Shunian Chen, Wenbo Zan, Peizhao Li, Mingxuan Wang, Dingjie Song, Bo Li, Yan Hu, and Benyou Wang. 2024. BlenderLLM: Training Large Language Models for Computer-Aided Design with Self-improvement. arXiv:2412.14203 [cs.HC] <https://arxiv.org/abs/2412.14203>
- [4] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). *arXiv preprint* (2025).
- [5] S. Farquhar et al. 2024. Detecting hallucinations in large language models using entropy-based uncertainty estimators. *Nature* (2024).
- [6] Shanshan Han, Qifan Zhang, Yuhang Yao, Weizhao Jin, and Zhaozhao Xu. 2025. LLM Multi-Agent Systems: Challenges and Open Problems. arXiv:2402.03578 [cs.MA] <https://arxiv.org/abs/2402.03578>
- [7] Ziniu Hu, Ahmet Iscen, Aashi Jain, Thomas Kipf, Yisong Yue, David A. Ross, Cordelia Schmid, and Alireza Fathi. 2024. SceneCraft: An LLM Agent for Synthesizing 3D Scene as Blender Code. arXiv:2403.01248 [cs.CV] <https://arxiv.org/abs/2403.01248>
- [8] Chenhan Jiang. 2024. A Survey On Text-to-3D Contents Generation In The Wild. arXiv:2405.09431 [cs.CV] <https://arxiv.org/abs/2405.09431>
- [9] Chenhan Jiang, Yihan Zeng, Hang Xu, and Dit-Yan Yeung. 2025. CoherenDream: Boosting Holistic Text Coherence in 3D Generation via Multimodal Large Language Models Feedback. arXiv:2504.19860 [cs.CV] <https://arxiv.org/abs/2504.19860>
- [10] Vijay R. Konda and John N. Tsitsiklis. 2000. Actor-Critic Algorithms. *Advances in Neural Information Processing Systems* 12 (2000).
- [11] Xinhang Liu, Chi-Keung Tang, and Yu-Wing Tai. 2025. WorldCraft: Photo-Realistic 3D World Creation and Customization via LLM Agents. arXiv:2502.15601 [cs.CV] <https://arxiv.org/abs/2502.15601>
- [12] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651 [cs.CL] <https://arxiv.org/abs/2303.17651>
- [13] Shalini Maiti, Lourdes Agapito, and Filippos Kokkinos. 2025. Gen3DEval: Using vLLMs for Automatic Evaluation of Generated 3D Objects. arXiv:2504.08125 [cs.CV] <https://arxiv.org/abs/2504.08125>
- [14] Richard S. Sutton and Andrew G. Barto. 2020. *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [15] Nan Xiang, Tianyi Liang, Haiwen Huang, Shiqi Jiang, Hao Huang, Yifei Huang, Liangyu Chen, Changbo Wang, and Chenhui Li. 2025. Sel3DCraft: Interactive Visual Prompts for User-Friendly Text-to-3D Generation. arXiv:2508.00428 [cs.GR] <https://arxiv.org/abs/2508.00428>
- [16] Wenkai Yang, Jingwen Chen, Yankai Lin, and Ji-Rong Wen. 2025. DeepCritic: Deliberate Critique with Large Language Models. arXiv:2505.00662 [cs.CL] <https://arxiv.org/abs/2505.00662>